

SmartTask

جزوه جامع فنی – بخش دوم

موضوعات: دسترسی‌ها | ساختار داده | نمودارها | تحلیل‌ها | SignalR | ایمیل | هوش مصنوعی | Push Notification

تاریخ تهیه: ۲۶ مرداد ۱۴۰۵

۱. سطوح دسترسی Public/Private فضاهای کاری

۱.۱ مفهوم

در هنگام ایجاد یک Workspace (فضای کاری)، کاربر می‌تواند یکی از دو حالت زیر را انتخاب کند:

حالت	نماد	توضیح
خصوصی (Private)		فقط اعضای دعوت‌شده می‌توانند ببینند
عمومی (Public)		همه کاربران سامانه می‌توانند ببینند و بپیوندند

۱.۲ پیاده‌سازی در کد

تعریف Enum:

```
// Models/Enums/VisibilityType.cs
public enum VisibilityType
{
    Private = 1,
    Public = 2
}
```

ذخیره در مدل Workspace:

```
// Models/Entities/Workspace.cs
public VisibilityType Visibility { get; set; } = VisibilityType.Private;
```

مقدار پیش‌فرض Private است.

۱.۳ منطق کنترل دسترسی

متد مرکزی بررسی عضویت در BaseController.cs :

```
protected async Task<bool> IsWorkspaceMemberAsync(
    ApplicationDbContext context, int workspaceId)
{
    return await context.ProjectMembers
        .AnyAsync(x => x.WorkspaceId == workspaceId
            && x.ApplicationUserId == CurrentUser.UserId
            && x.ViewState);
}
```

۱.۴ نکته مهم برای دفاع

در نسخه فعلی، مقدار **Visibility** در دیتابیس ذخیره و در UI نمایش داده می‌شود. کنترل دسترسی واقعی بر اساس **عضویت** کاربر در **Workspace** است. برای نسخه‌های آینده، می‌توان قابلیت "مشاهده Workspace های عمومی توسط غیراعضا" و "پیوستن بدون دعوت" را اضافه کرد.

۲. ارتباط Backlog → Sprint → User Story → Task

۲.۱ ساختار سلسله‌مراتبی

SmartTask از مدل **Scrum** برای مدیریت تسک‌ها استفاده می‌کند:

```

Workspace (فضای کاری)
├── Project (پروژه)
│   ├── Backlog (فقط یکی به ازای هر پروژه)
│   │   ├── UserStory (آیتم‌های بک‌لاگ)
│   │   │   ├── Task (وظایف اجرایی)
│   │   │   └── SubTask (زیرتسک‌ها)
│   └── Sprint (اسپرینت‌ها)
│       ├── UserStory (آیتم‌های منتقل‌شده از بک‌لاگ)
│       └── OffroadTask (کارهای خارج از دامنه)

```

۲.۲ توضیح هر لایه

Backlog (فقط یکی)

هر پروژه دقیقاً یک Backlog دارد. نام پیش فرض: Product Backlog. نقش: محل ذخیره User Story هایی که هنوز به اسپرینتی تخصیص نداده‌اید.

User Story

زیرمجموعه Backlog (فیلد BacklogId). می‌تواند به Sprint تخصیص داده شود (فیلد SprintId که nullable است).

- اگر SprintId = null → هنوز در بک‌لاگ منتظر است
- فیلدهای کلیدی: StoryPoint , BusinessValue , Priority , Status

Task

زیرمجموعه مستقیم UserStory (فیلد UserStoryId). هیچ ارتباط مستقیمی با Sprint ندارد.

- فیلدهای کلیدی: Estimate (ساعت), StartDate , DueDate , Status , Type , Priority

SubTask

زیرمجموعه Task (فیلد TaskItemId). فقط Title و IsCompleted دارد.

OffroadTask

مستقیماً به Project وصل است. برای کارهای خارج از دامنه رسمی پروژه.

۲.۳ فرآیند کاری

1. ایجاد User Story در (SprintId = null) Backlog
2. انتقال User Story به Sprint (اسپرینت پلنینگ)
3. ایجاد Task زیر User Story
4. ایجاد SubTask زیر Task (اختیاری)

۲.۴ مثال عملی

Backlog: Product Backlog

- └─ User Story: "ورود کاربر به سامانه"
 - └─ Sprint: "اسپرینت ۱"
 - └─ StoryPoint: 5
 - └─ Task ها:
 - └─ Task: "طراحی فرم ورود"
 - └─ SubTask: "ایمیل input ایجاد"
 - └─ SubTask: "رمز عبور input ایجاد"
 - └─ SubTask: "دکمه ورود"
 - └─ Task: "اعتبارسنجی سمت سرور"
 - └─ Task: "اتصال به دیتابیس"
 - └─ Task: "تست ورود"

۲.۵ پاسخ به سوال رایج

نوع کار	کجا تعریف کنید
کار رسمی پروژه	User Story → Backlog → Sprint
زیرکار یک User Story	Task (زیر User Story)
جزئیات یک Task	SubTask (زیر Task)
کار ضروری خارج از پروژه	OffroadTask (مستقل)

۳. نمودار Velocity و Burndown

۳.۱ نمودار Burndown (نرود کار)

فایل: `Services/Implementations/SprintService.cs` → متد `GetBurndownDataAsync`

نمایش گرافیکی پیشرفت کار در طول یک اسپرینت، بر اساس **Story Point** محاسبه می‌شود.

فرمول:

(تعداد_کل_روزها × روز_های_سپری_شده / TotalPoints - IdealRemaining = خط ایده آل
 های تکمیل‌شده تا آن روز SP مجموع - ActualRemaining = خط واقعی

مثال عددی (اسپرینت ۱۰ روزه، ۴۰ Story Point):

روز	خط ایده آل	خط واقعی	توضیح
روز ۱	۴۰	۴۰	شروع
روز ۳	۳۲	۳۵	کندتر ⚠️
روز ۵	۲۴	۲۰	جلوتر ✅
روز ۱۰	۰	۵	۵ امتیاز مانده

۳.۲ نمودار Velocity (سرعت)

فایل: `SprintService.cs` → متد `GetVelocityDataAsync`

مقایسه کار برنامه‌ریزی‌شده با کار واقعی تکمیل‌شده در اسپرینت‌های گذشته.

PlannedPoints = های اسپرینت UserStory تمام StoryPoint مجموع
 CompletedPoints = های UserStory فقط StoryPoint مجموع

۳.۳ مقایسه دو نمودار

ویژگی	Burndown	Velocity
مقطع زمانی	یک اسپرینت	چند اسپرینت گذشته
نوع نمودار	خطی (Line)	میله‌ای (Bar)
هدف	پیشرفت لحظه‌ای	روند کلی عملکرد
کتابخانه رسم	Chart.js 4.4.0	

۴. تحلیل بار کاری (Workload Analysis)

فایل: WorkloadAnalysisService.cs | کنترلر: WorkloadController

۴.۱ فرمول

$AssignedHours = \text{تعداد اعضای تخصیص یافته} \div \text{Task هر Estimate} \times \text{مجموع}$
 $UtilizationPercent = (AssignedHours \div WeeklyCapacityHours) \times 100$

۴.۲ سه سطح وضعیت

سطح	شرط	رنگ	معنی
under	کمتر از ۸۰٪	سبز ●	ظرفیت خالی دارد
balanced	۸۰٪ تا ۱۰۰٪	زرد ●	بهینه
overloaded	بیشتر از ۱۰۰٪	قرمز ●	اضافه بار ⚠️

۴.۳ نمایش دوگانه

- بارکاری کل پروژه → تمام Task های باز
- بارکاری اسپرینت فعال → فقط Task های اسپرینت جاری

۵. پیش بینی ریسک تأخیر (Delay Risk)

فایل: DelayRiskService.cs | کنترلر: RiskController

۵.۱ چهار شاخص

شاخص	وزن	حداکثر
۱. نسبت Task های عقب افتاده	40%	۴۰ امتیاز
۲. نسبت اعضای اضافه بار	30%	۳۰ امتیاز
۳. زنجیره های پرریسک	20%	۲۰ امتیاز
۴. فعالیت اخیر Cascade	10%	۱۰ امتیاز

۵.۲ فرمول

```
OverdueScore = (تعداد عقب افتاده ÷ کل باز) × 40
WorkloadScore = (کل اعضا ÷ overloaded تعداد) × 30
DependencyScore = min(تعداد زنجیره ها × 4, 20)
CascadeScore = min(های اخیر × 2, cascade10 تعداد)
RiskScore = min(مجموع, 100)
```

۵.۳ سطوح ریسک

امتیاز	سطح	رنگ
۰-۲۵	کم	 سبز
۲۶-۵۰	متوسط	 زرد
۵۱-۷۵	بالا	 نارنجی
۷۶-۱۰۰	بحرانی	 قرمز

۵.۴ تحلیل متنی با AI

با کلیک روی دکمه "تحلیل هوشمند"، داده های عددی به LM Studio (Gemma) ارسال و تحلیل روایی فارسی تولید می شود.

۶. شاخص سلامت پروژه (Project Health Score)

فایل: ProjectHealthService.cs | کنترلر: HealthController

۶.۱ چهار زیرشاخص

زیرشاخص	وزن	منبع
سلامت زمان بندی	30%	درصد Task های به موقع
سلامت بارکاری	25%	درصد اعضای overloaded
سلامت وابستگی ها	25%	زنجیره های پرریسک
پیشرفت واقعی	20%	درصد تکمیل UserStory

۶.۲ ارتباط بین سه تحلیل

```
Workload Analysis → Delay Risk → Project Health
Dependency Service └─┬─┘
                     └─┬─┘ AI Narrative
```

۷. سرویس SignalR – ارتباط بلادرنگ

۷.۱ SignalR چیست؟

کتابخانه میکروسافت برای ارتباط بلادرنگ (Real-Time) بین سرور و کلاینت. پروتکل ها: WebSocket (ترجیحی)، Server-Sent Events، Long Polling.

۷.۲ چرا SignalR؟

- اعلان جدید رسید → بدون رفرش صفحه نمایش داده شود

- پیام چت جدید → بلادرنگ دریافت شود
- کسی آنلاین/آفلاین شد → همه بفهمند
- کسی تایپ می‌کند → نشان داده شود

۷.۳ دو Hub

Hub اول: NotificationHub

فایل: Hubs/NotificationHub.cs | آدرس: /hubs/notification

هر کاربر به گروه `user-{userId}` اضافه می‌شود و فقط اعلان‌های خودش را دریافت می‌کند.

Hub دوم: ChatHub

فایل: Hubs/ChatHub.cs | آدرس: /hubs/chat

متد	عملکرد	رویداد
<code>SendMessage()</code>	ارسال پیام	<code>ReceiveMessage</code>
<code>EditMessage()</code>	ویرایش	<code>MessageEdited</code>
<code>DeleteMessage()</code>	حذف	<code>MessageDeleted</code>
<code>Typing()</code>	تایپ	<code>UserTyping</code>
<code>ToggleReaction()</code>	واکنش	<code>ReactionToggled</code>
<code>TogglePin()</code>	پین	<code>PinToggled</code>

۷.۴ ثبت در Program.cs

```
builder.Services.AddSignalR(); // خط 63
app.MapHub<NotificationHub>("/hubs/notification"); // خط 207
app.MapHub<ChatHub>("/hubs/chat"); // خط 209
```

۷.۵ جریان کامل (مثال: اعلان تغيير وضعیت تسک)

۱. وضعیت تسک را تغيير می‌دهد A کاربر
↓
۲. TaskController تغيير را ذخيره می‌کند
↓
۳. NotificationService.CreateAsync()
↓
۴. اعلان در دیتابیس ذخيره
↓
۵. IHubContext → SendAsync("ReceiveNotification", ...)
↓
۶. SignalR به مرورگر کاربر WebSocket از طريق
↓
۷. آپدیت می‌شود badge جاوا اسکریپت
↓
!بدون رفرش صفحه 🛎

۷.۶ فایل‌های کلیدی

فایل	نقش
Program.cs	ثبت سرویس و Hubها
Hubs/NotificationHub.cs	هاب اعلان‌ها
Hubs/ChatHub.cs	هاب چت + آنلاین/آفلاین
NotificationService.cs	ارسال اعلان از سرور
IPresenceTracker.cs	ردیابی وضعیت اتصال
wwwroot/js/notification.js	کلاینت اعلان‌ها
wwwroot/js/chat.js	کلاینت چت

۸. سرویس ایمیل – Gmail SMTP

۸.۱ پیکربندی

پارامتر	مقدار
Host	smtp.gmail.com
Port	(STARTTLS) 587
SSL	فعال
اعتبارنامه	ایمیل Gmail + App Password

۸.۲ چرا Gmail SMTP؟

- رایگان (تا ۵۰۰ ایمیل در روز)
- قابل اطمینان (سرور گوگل)
- بدون نیاز به سرور اختصاصی

۸.۳ معماری سرویس

```
Services/Email/  
├── IEmailService.cs    ← اینترفیس  
├── EmailService.cs     ← پیاده سازی  
└── EmailSettings.cs   ← تنظیمات
```

۸.۴ پیاده‌سازی

```
public async Task SendEmailAsync(string to, string subject, string body)
{
    using var client = new SmtpClient(_settings.Host, _settings.Port);
    client.EnableSsl = _settings.EnableSsl;
    client.Credentials = new NetworkCredential(_settings.Email, _settings.Password);
    var message = new MailMessage
    {
        From = new MailAddress(_settings.Email),
        Subject = subject,
        Body = body,
        IsBodyHtml = true
    };
    message.To.Add(to);
    await client.SendMailAsync(message);
}
```

۸.۵ کجاها ایمیل ارسال می‌شود؟

#	مورد	فایل	موضوع
۱	تایید ایمیل ثبت‌نام	AccountController.cs	"تایید ایمیل SmartTask"
۲	بازیابی رمز عبور	AccountController.cs	"بازیابی رمز عبور"
۳	دعوت به Workspace	WorkspaceInvitationService.cs	"دعوت به فضای کاری"
۴	تست ایمیل	AccountController.cs	"SmartTask Test"

۸.۶ جریان کامل (دعوتنامه)

```
۱. مدیر "دعوت عضو جدید" را کلیک می‌کند
↓
۲. WorkspaceInvitationService.InviteAsync()
↓
۳. رکورد دعوت در دیتابیس (توکن + انقضای ۳ روز)
↓
۴. _emailService.SendEmailAsync()
↓
۵. SmtpClient → smtp.gmail.com:587
↓
۶. با لینک دعوت ارسال HTML ایمیل
↓
۷. کاربر لینک را کلیک می‌کند → صفحه ثبت‌نام/پذیرش
```

۹. سرویس هوش مصنوعی – LM Studio

۹.۱ LM Studio چیست؟

نرم‌افزار محلی برای اجرای مدل‌های زبانی (LLM) روی سرور خودتان. داده‌ها به خارج نمی‌رود و هزینه‌ای ندارد.

۹.۲ پیکربندی

پارامتر	مقدار
آدرس	<code>http://92.246.145.99:1234/v1/chat/completions</code>
مدل	<code>google/gemma-4-12b</code>
فرمت API	سازگار با OpenAI

۹.۳ چرا LM Studio و نه OpenAI؟

معیار	OpenAI	LM Studio
هزینه	پولی	رایگان
حریم خصوصی	داده به خارج	محلی
محدودیت	Rate Limit	بدون محدودیت

۹.۴ اینترفیس

```
public interface IAIClientService
{
    Task<string> GetCompletionAsync(
        string systemPrompt,    // دستور سیستم
        string userPrompt,      // درخواست کاربر
        double temperature = 0.7, // خلاقیت (0=دقیق، 1=خلاقانه)
        CancellationToken cancellationToken = default);
}
```

۹.۵ کجاها استفاده می‌شود؟

#	سرویس	فایل	Temperature	کاربرد
۱	تجزیه تسک	TaskBreakdownService.cs	0.5	تقسیم تسک به زیرتسک
۲	گزارش اسپرینت	SprintReportAiService.cs	0.7	تولید گزارش روایی
۳	تحلیل ریسک	DelayRiskService.cs	0.6	تحلیل متنی ریسک

۹.۶ چرا temperature های متفاوت؟

- تجزیه تسک (0.5): نیاز به دقت، پاسخ‌ها منطقی باشند
- گزارش اسپرینت (0.7): روایی و خوانا باشد
- تحلیل ریسک (0.6): ترکیب دقت عددی و خوانایی

۹.۷ جریان کامل (تجزیه تسک)

```
۱. "کاربر: "پایاده سازی صفحه ورود
↓
۲. AI کلیک "تجزیه هوشمند با
↓
۳. TaskBreakdownService → GetCompletionAsync()
↓
۴. POST به LM Studio
↓
۵. پاسخ:
[{"title": "طراحی فرم", "estimate": 3},
{"title": "اعتبارسنجی", "estimate": 2}]
↓
۶. کاربر تایید می کند
```

۱۰. سرویس Webpushr – Push Notification

۱۰.۱ Push Notification چیست؟

پیامی که خارج از مرورگر و حتی وقتی صفحه بسته است نمایش داده می شود.

۱۰.۲ تفاوت با SignalR

ویژگی	SignalR	Webpushr
شرط نمایش	مرورگر باز	حتی بسته
نوع ارتباط	WebSocket	HTTP Push
کاربرد	چت + اعلان درون برنامه ای	اعلان خارج از مرورگر

۱۰.۳ آدرس API

`https://api.webpushr.com/v1/notification/send/sid`

رایگان تا ۱۰,۰۰۰ مشترک.

۱۰.۴ سه متد اصلی

کاربرد	متد
اعلان پیام چت	<code>SendChatMessagePushAsync()</code>
اعلان آزمایشی	<code>SendTestPushAsync()</code>
ارسال HTTP به API	<code>SendWebpushrNotification()</code>

۱۰.۵ جریان کامل (پیام چت)

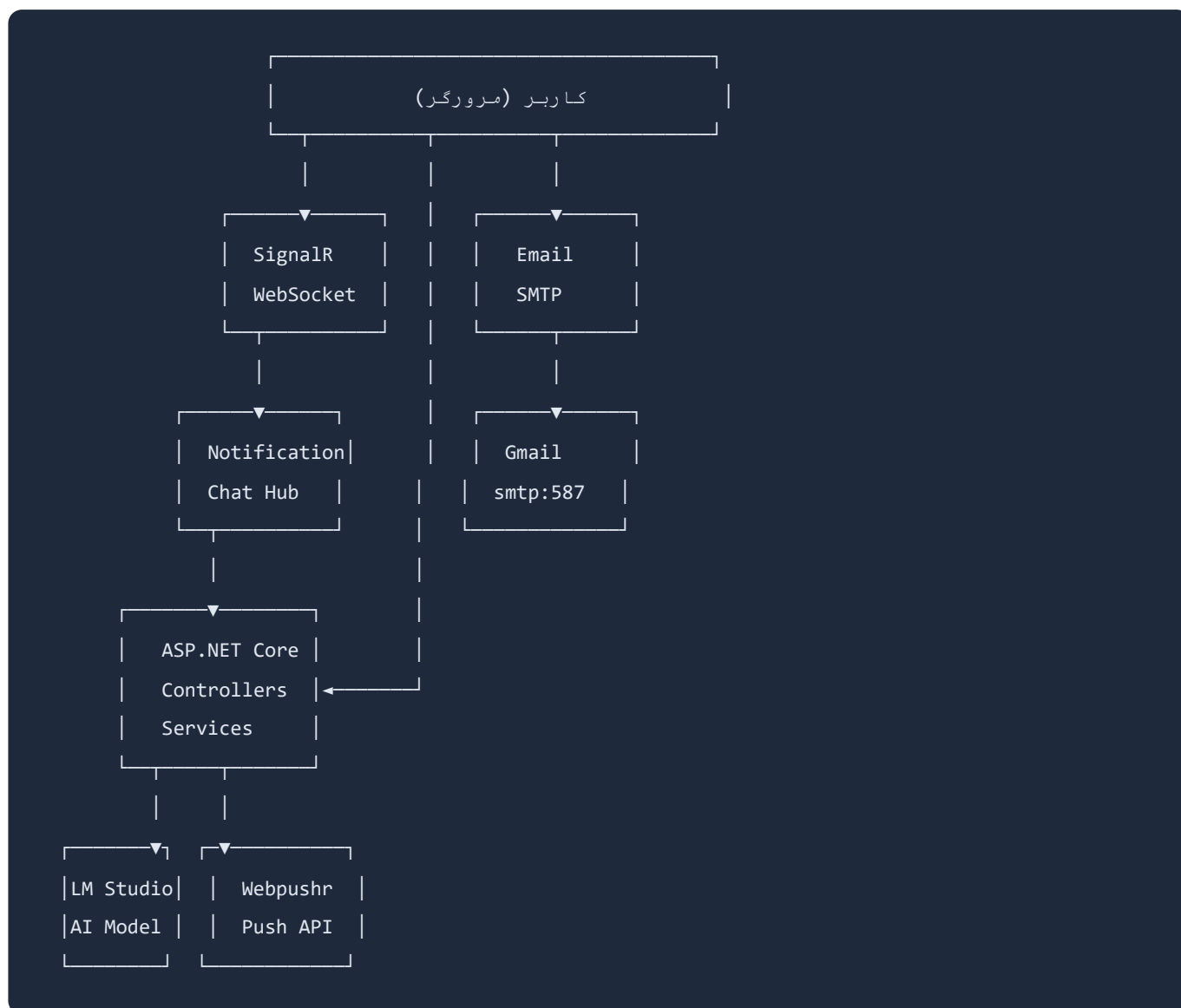
- پیام ارسال می‌کند A کاربر ۱.
↓
- `ChatHub.SendMessage()` ۲.
↓
- پیام در دیتابیس ذخیره ۳.
↓
- کاربران آنلاین (بلادرنگ) → `SignalR` ۴.
 کاربران آفلاین (Push) → `Webpushr`
↓
- POST به `api.webpushr.com` ۵.
↓
- ! اعلان در ویندوز/موبایل ۶. 📢

۱۱. نمودار جامع و مقایسه نهایی

۱۱.۱ مقایسه سه سرویس خارجی

ویژگی	Email (SMTP)	AI (LM Studio)	Webpushr
پروتکل	SMTP	HTTP REST	HTTP REST
آدرس	smtp.gmail.com:587	92.246.145.99:1234	api.webpushr.com
هزینه	رایگان	رایگان	رایگان
کاربرد	ایمیل رسمی	تحلیل و تولید متن	اعلان فوری
سرعت	۱-۵ ثانیه	۵-۶۰ ثانیه	> ۱ ثانیه
فایل‌ها	*/Services/Email	*/Services/AI	WebpushrService.cs

۱۱.۲ نمودار جریان داده



۱۱.۳ تعداد خطوط کد

بخش	فایلها	خطوط کد
SignalR	Hub ها + کلاینتها	۱,۱۵۰~
Email	*/Services/Email	۶۰~
AI	*/Services/AI	۱۰۰~
Webpushr	WebpushrService.cs	۱۵۰~
مجموع		۱,۴۶۰~

نکته پایانی: تمام این سرویس‌ها با استفاده از الگوی **Dependency Injection** متصل شده‌اند. این الگو باعث می‌شود: تست‌پذیری بالا، تعویض آسان سرویس‌ها، و کد تمیز.